

PCA

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import seaborn as sns
import numpy as np

# =====
# 1) CITIRE DATE + TRATARE VALORI LIPSA
# =====

# Citim setul de date; index_col=0 => prima coloană devine index (nume
# observații / ID-uri)
data = pd.read_csv("ConsumAlimentar.csv", index_col=0)

# Imputare: înlocuim valorile lipsă (NaN) cu media coloanei (doar pentru
# variabile numerice)
data = data.apply(lambda col: col.fillna(col.mean()), axis="rows")

# Extragem valorile numerice în format NumPy (matrice n x p)
data_numeric = data.values

# =====
# 2) STANDARDIZARE (NECESARĂ ÎN PCA)
# =====

# Standardizare: fiecare variabilă -> medie 0, deviație standard 1
# Motiv: PCA este sensibil la scale. Variabilele cu valori mari ar domina
# rezultatul.
data_numeric = StandardScaler().fit_transform(data_numeric)

# =====
# 3) PCA INITIAL (PENTRU EIGENVALUES ȘI VARIANTA EXPLICATĂ)
# =====

# PCA fără a fixa numărul de componente (calculăm toate componentele
# posibile)
pca = PCA()
pca.fit(data_numeric)

# Eigenvalues (valoare proprie) = varianța explicată de fiecare componentă
# principală
eigenvalues = pca.explained_variance_

# Procent din varianța totală explicat de fiecare componentă
eigen_values_ratio = pca.explained_variance_ratio_ * 100

# Procent cumulativ (suma procentelor până la componenta k)
cumsum = np.cumsum(eigen_values_ratio)

# Tabel pentru raport: eigenvalues + procente + cumul
df_varianța = pd.DataFrame({
    "Valoare Proprie": eigenvalues,
    "Varianta explicata %": eigen_values_ratio,
    "Varianta cumulata %": cumsum
})
```

```

}, index=[f"C{i+1}" for i in range(len(eigenvalues))])

print(df_varianta)

# =====
# 4) SCREE PLOT + CRITERII DE SELECTIE A NR. DE COMPONENTE
# =====

def scree_plot(eigenvalues, threshold=79):
    """
    Scree plot + 3 criterii uzuale de selecție:
    1) Kaiser: păstrăm componentele cu eigenvalue > 1
    2) Prag de varianță cumulată: păstrăm primele componente până depășim
    threshold%
    3) Cattell (cot): punctul unde scăderea eigenvalues devine mică
    (euristic)

    Returnează:
    - kaiser: numărul de componente conform Kaiser
    - cattell: aproximare a componentelor până la "cot"
    - thres: numărul de componente până la pragul de varianță
    """
    plt.title("Scree Plot (eigenvalues)")

    m = len(eigenvalues)
    indices = np.arange(1, m + 1)

    # Curba eigenvalues
    plt.plot(indices, eigenvalues, "r-", label="Valori proprii")

    # Linie Kaiser (eigenvalue = 1)
    plt.axhline(y=1, color="b", linestyle="--", label="Kaiser ( $\lambda = 1$ )")

    # Prag de varianță cumulată calculat din eigenvalues
    # (echivalent cu explained_variance_ratio, doar că îl recalculăm
    explicit)
    thres_praguri = np.cumsum(eigenvalues) * 100 / np.sum(eigenvalues)
    thres = np.where(thres_praguri > threshold)[0][0] + 1

    # Marcăm pe grafic componenta unde depășim threshold%
    if thres <= m:
        plt.axhline(
            y=eigenvalues[thres - 1],
            color="y",
            linestyle="--",
            label=f"Prag varianță cumulată > {threshold}%"
        )

    # "Cotul" (Cattell) printr-o regulă simplificată:
    # când diferența dintre valori consecutive devine mică (aproape 0),
    # considerăm că graficul se aplatizează.
    cattell_vec = np.where(np.diff(eigenvalues) > -0.1)[0]
    cattell = cattell_vec[0] + 1 if len(cattell_vec) > 0 else None

    if cattell is not None:
        plt.axhline(
            y=eigenvalues[cattell - 1],
            color="g",
            linestyle="--",
            label="Cattell (zona de aplatizare)"

```

```

    )

    # Număr de componente după Kaiser
    kaiser = np.sum(eigenvalues > 1)

    plt.xlabel("Componentă")
    plt.ylabel("Valoare proprie ( $\lambda$ )")
    plt.legend()
    plt.show()

    return kaiser, cattell, thres

# Calculăm numărul de componente recomandat de fiecare criteriu
kaiser, cattel, thres = scree_plot(eigenvalues, threshold=79)

# =====
# 5) PCA FINAL (CU NR. DE COMPONENTE ALES)
# =====

# Refacem PCA doar cu numărul de componente ales.
# Aici păstrăm varianta "thres" (prag de varianță cumulată).
pca = PCA(n_components=thres)
pca.fit(data_numeric)

# =====
# 6) INCARCATURI FACTORIALE (LOADINGS)
# =====

# Loadings = corelația dintre variabilele originale și componente
# O formulă uzuală: loadings = components_.T * sqrt(eigenvalues)
# (după convenția PCA pe date standardizate)
loadings = pca.components_.T * np.sqrt(eigenvalues[:thres])

df_loadings = pd.DataFrame(
    loadings,
    columns=[f"C{i+1}" for i in range(loadings.shape[1])],
    index=data.columns
)
print(df_loadings)

# =====
# 7) HEATMAP PENTRU LOADINGS (INTERPRETARE RAPIDA)
# =====

def corelograma_loadings(df_loadings):
    plt.title("Corelograma încărcăturilor (loadings)")
    sns.heatmap(df_loadings, center=0, cmap="RdBu", annot=True)
    plt.show()

corelograma_loadings(df_loadings)

# =====
# 8) CERCUL CORELAȚIILOR (PLANUL C1-C2)
# =====

def cerc_corelatii(df_loadings, c1=0, c2=1):

```

```

"""
Reprezentăm variabilele ca vectori în planul (C1, C2).
Direcția și lungimea vectorului sugerează asocierea cu componentele.
"""
x = df_loadings.iloc[:, c1]
y = df_loadings.iloc[:, c2]

plt.title("Cerc corelații (variabile în planul componentelor)")

# Cerc unitate (orientativ)
cerc = plt.Circle((0, 0), 1, fill=False, color="gray")
plt.gca().add_patch(cerc)

plt.axhline(y=0, color="gray", linewidth=0.8)
plt.axvline(x=0, color="gray", linewidth=0.8)

# Vectori pentru fiecare variabilă
for i, nume in enumerate(df_loadings.index):
    plt.arrow(
        0, 0, x[i], y[i],
        color="r",
        length_includes_head=True,
        head_width=0.09,
        alpha=0.7
    )
    plt.text(x[i] * 1.06, y[i] * 1.06, nume, fontsize=9)

plt.xlim(-1.1, 1.1)
plt.ylim(-1.1, 1.1)
plt.gca().set_aspect('equal', 'box')
plt.xlabel(f"C{c1 + 1}")
plt.ylabel(f"C{c2 + 1}")
plt.show()

# Cerc corelații pentru primele 2 componente
cerc_corelatii(df_loadings, 0, 1)

# =====
# 9) SCORURI PCA (COORDONATELE OBSERVATIILOR IN SPATIUL COMPONENTELOR)
# =====

# Scoruri = proiecția observațiilor în spațiul componentelor
scores = pca.transform(data_numeric)

df_scores = pd.DataFrame(
    scores,
    columns=[f"C{i+1}" for i in range(scores.shape[1])],
    index=data.index
)
print(df_scores)

# =====
# 10) GRAFIC OBSERVATII IN PLANUL COMPONENTELOR (C1-C2)
# =====

def plot_componente_scoruri(df_scores, c1=0, c2=1):
    plt.title("Observații în planul componentelor (scoruri)")
    plt.scatter(df_scores.iloc[:, c1], df_scores.iloc[:, c2], color="b")

```

```

plt.axhline(y=0, linewidth=0.8, color="gray")
plt.axvline(x=0, linewidth=0.8, color="gray")

plt.xlabel(f"C{c1 + 1}")
plt.ylabel(f"C{c2 + 1}")

# Etichetăm punctele cu numele observațiilor (indexul)
for idx in df_scores.index:
    x_val = df_scores.loc[idx, f"C{c1 + 1}"]
    y_val = df_scores.loc[idx, f"C{c2 + 1}"]
    plt.text(x_val, y_val, str(idx), fontsize=8)

plt.show()

plot_componente_scoruri(df_scores, 0, 1)

# =====
# 11) INDICATORI DE CALITATE: COSINUSURI, COMUNALITATI, CONTRIBUTII
# =====

def cosinusuri_comunalitati_contributii(df_loadings, df_scores):
    """
    - Cosinusuri^2 (pentru variabile): loadings^2
      indică cât de bine este reprezentată variabila pe fiecare componentă.

    - Comunalități (pentru variabile): suma cos^2 pe componente
      indică proporția din varianta variabilei explicată de componentele
      păstrate.

    - Contribuții (pentru observații): (score^2) / (suma score^2 pe
      componentă)
      indică ce observații contribuie cel mai mult la o componentă.
    """
    # Cosinusuri^2 pentru variabile
    cos = df_loadings ** 2

    # Comunalități pentru variabile
    comunalitati = pd.DataFrame(cos.sum(axis="columns"),
                                columns=["Comunalitati"])

    # Contribuțiile observațiilor la componente:
    # (IMPORTANT: împărțirea se face pe coloană, adică pe fiecare
    # componentă separat)
    contributii = (df_scores ** 2) / (df_scores ** 2).sum(axis=0)

    return cos, comunalitati, contributii

cos, comunalitati, contributii =
cosinusuri_comunalitati_contributii(df_loadings, df_scores)

print("\nCOSINUS^2 (variabile x componente):\n", cos)
print("\nCOMUNALITATI (pe variabile):\n", comunalitati)
print("\nCONTRIBUTII (observatii la componente):\n", contributii)

# =====
# 12) HEATMAP PENTRU COMUNALITATI
# =====

```

```
def corelograma_comunalitati(comunalitati):
    plt.title("Comunalități (calitatea reprezentării variabilelor)")
    sns.heatmap(comunalitati, annot=True, cmap="YlGnBu")
    plt.show()

corelograma_comunalitati(comunalitati)
```

Factoriala

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from factor_analyzer import FactorAnalyzer, calculate_bartlett_sphericity,
calculate_kmo

# =====
# 1) CITIRE DATE + TRATARE VALORI LIPSA + STANDARDIZARE
# =====

# Citim setul de date; index_col=0 presupune ca prima coloană este un ID /
# etichetă.
data = pd.read_csv("ConsumAlimentar.csv", index_col=0)

# Înlocuim valorile lipsă (NaN) cu media pe coloană (imputare simplă).
# axis="rows" înseamnă că aplicăm funcția pe fiecare coloană (nu pe
# rânduri).
data = data.apply(lambda col: col.fillna(col.mean()), axis="rows")

# Standardizare: transformăm fiecare variabilă astfel încât să aibă medie 0
# și deviație standard 1.
# E important pentru FA, ca variabilele să fie comparabile.
data_transf = StandardScaler().fit_transform(data)

# =====
# 2) TESTE DE ADECVARE PENTRU ANALIZA FACTORIALA (FA)
# =====

# Testul Bartlett verifică dacă matricea de corelații este semnificativ
# diferită de identitatea.
# Dacă p-value < 0.05 => există corelații suficiente => FA e justificată.
chi_square_value, p_value = calculate_bartlett_sphericity(data_transf)
print(f"Bartlett Test: {chi_square_value}, p-value:{p_value}\n")

# Testul KMO (Kaiser-Meyer-Olkin) măsoară dacă datele sunt potrivite pentru
# FA.
# Reguli uzuale:
# KMO < 0.5 slab
# 0.5-0.6 mediocru
# 0.6-0.7 acceptabil
# 0.7-0.8 bun
# >0.8 foarte bun
kmo_all, kmo_model = calculate_kmo(data_transf)
print(f"Testul KMO: {kmo_model}\n")

# =====
```

```

# 3) EXTRAGEREA VALORILOR PROPRII (EIGENVALUES) PENTRU A ALEGE NR. DE FACTORI
# =====

# Inițial "fit" cu numărul maxim de factori (=nr. variabile), ca să putem
# obține eigenvalues.
# rotation="varimax" => rotație ortogonală pentru interpretare mai clară.
fa = FactorAnalyzer(n_factors=data_transf.shape[1], rotation="varimax")
fa.fit(data_transf)

# Eigenvalues = varianța explicată de fiecare factor (în ordinea
# factorilor).
eigenvalues = fa.get_eigenvalues()[0]

# Procent din varianță explicată de fiecare factor
var_percent = eigenvalues * 100 / np.sum(eigenvalues)

# Varianță cumulată (câtă varianță e explicată de primii k factori)
var_cum = np.cumsum(var_percent)

# Construim un tabel cu rezultatele pentru raport / interpretare.
tabel_varianta = pd.DataFrame({
    "Valoare Proprie": eigenvalues,
    "% Varianta Explicata": var_percent,
    "% Varianta Cumulata": var_cum
}, index=[f"F{i+1}" for i in range(len(eigenvalues))])

# =====
# 4) SCREE PLOT + CRITERII DE SELECTIE A FACTORILOR
# =====

def scree_plot(eigenvalues, var_cum, threshold=79):
    """
    Desenează:
    - Scree plot: eigenvalues vs. număr factor
    - Linie Kaiser (eigenvalue=1)
    - Prag de varianță cumulată (ex: 79%)
    - O aproximare a "cotului" (Cattell), folosind o regulă simplificată

    Return:
    - kaiser: nr. factori cu eigenvalue > 1
    - thres: primul k unde varianța cumulată depășește threshold (%)
    - cattell: aproximare a punctului unde graficul se aplatizează
    """
    plt.title("Plot varianta componente plus criterii de relevanta")

    m = len(eigenvalues)
    indices = np.arange(1, m + 1)

    # Linie eigenvalues
    plt.plot(indices, eigenvalues, "r-", label="Valoare proprie factori")

    # Criteriul Kaiser: păstrăm factorii cu eigenvalue > 1
    plt.axhline(y=1, color="b", linestyle="--", label="Kaiser")

    # Prag de varianță cumulată: găsim primul k care depășește threshold (%)
    thres = np.where(var_cum > threshold)[0][0] + 1
    if thres <= m:
        plt.axhline(

```

```

        y=eigenvalues[thres - 1],
        color="y",
        linestyle="--",
        label=f"Unde trece de {threshold}%"
    )

    # Cattell (cot): aici e o aproximare.
    # np.diff(eigenvalues) calculează scăderile între eigenvalues consecutive.
    # Când scăderea devine mică (aproape 0), graficul "se aplatizează".
    cattell_index = np.where(np.diff(eigenvalues) > -0.1)[0]
    cattell = cattell_index[0] + 1 if len(cattell_index) > 0 else None
    if cattell is not None:
        plt.axhline(
            y=eigenvalues[cattell - 1],
            color="g",
            linestyle="--",
            label="Unde se aplatizeaza graficul"
        )

    kaiser = np.sum(eigenvalues > 1)

    plt.legend()
    plt.show()

    return kaiser, thres, cattell

# Apelăm funcția de selecție și tipărim valorile obținute.
kaiser, thres, cattell = scree_plot(eigenvalues, var_cum)
print(f"Kaiser: {kaiser}\n thres: {thres}\n cattell:{cattell}\n")

# =====
# 5) REFACEM ANALIZA CU NR. DE FACTORI SELECTAT (thres)
# =====

# method="principal" => extracție prin principal factor / principal axis
# (în funcție de librărie)
# rotation="varimax" => rotație pentru interpretare clară
fa = FactorAnalyzer(n_factors=thres, rotation="varimax",
method="principal")
fa.fit(data_transf)

# Încărcături factoriale: corelația dintre variabile și factori (după
# rotație).
loadings = pd.DataFrame(
    fa.loadings_,
    index=data.columns,
    columns=[f"F{i+1}" for i in range(fa.loadings_.shape[1])]
)

print(tabel_varianta)
print("\nÎncărcăturile factoriale:\n", loadings.head())

# =====
# 6) CORELOGRAMA INCARCATURILOR FACTORIALE (HEATMAP)
# =====

def corelograma_corelatii(loadings):

```

```

plt.figure(figsize=(10, 7))
sns.heatmap(loadings, cmap="RdBu", center=0, annot=True)
plt.title("Corelograma Încărcăturilor Factoriale")
plt.show()

corelograma_corelatii(loadings)

# =====
# 7) CERCUL CORELAȚIILOR (PLOT PE 2 FACTORI)
# =====

def cerc_corelatii(loadings, c1=0, c2=1):
    """
    Reprezentăm variabilele ca vectori în planul (Factor c1, Factor c2).
    Dacă o variabilă are încărcături mari pe un factor, va "arăta" în
    direcția lui.
    """
    x = loadings.iloc[:, c1]
    y = loadings.iloc[:, c2]

    plt.title("Cerc Corelatii")

    # Cerc de rază 1: orientativ (în PCA are interpretare clasică, în FA e
    o vizualizare utilă).
    circle = plt.Circle((0, 0), 1, color="gray", linestyle="--",
fill=False)
    plt.gca().add_patch(circle)

    # Axe
    plt.axhline(y=0, color="gray", linewidth=0.8)
    plt.axvline(x=0, color="gray", linewidth=0.8)

    # Vectori pentru fiecare variabilă
    for i, varname in enumerate(loadings.index):
        plt.arrow(
            0, 0, x[i], y[i],
            head_width=0.09,
            color="r",
            alpha=0.7,
            length_includes_head=True
        )
        plt.text(x[i], y[i], varname, fontsize=9)

    # Limite și aspect pătrat
    plt.xlim(-1.1, 1.1)
    plt.ylim(-1.1, 1.1)
    plt.gca().set_aspect('equal', 'box')

    # Etichete (în codul tău scrie C1/C2, deși sunt factori; am păstrat
    stilul tău)
    plt.xlabel(f"C{c1+1}")
    plt.ylabel(f"C{c2+1}")

    plt.show()

cerc_corelatii(loadings, 0, 1)

# =====
# 8) COMUNALITATI + VARIANTA SPECIFICA

```

```

# =====

# Comunalitate = cât din varianta variabilei e explicată de factorii comuni
comunalitati = pd.DataFrame(
    fa.get_communalities(),
    index=loadings.index,
    columns=["Comunalitati"]
)

# Varianta specifică = 1 - comunalitate (partea neexplicată de factorii comuni)
varianta_specifica = pd.DataFrame(
    1 - comunalitati.values,
    index=comunalitati.index,
    columns=["Varianta Specifica"]
)

print(comunalitati)
print(varianta_specifica)

def corelograme_comunalitati_variante_specifice(comunalitati,
varianta_specifica):
    plt.title("Corelograma Comunalitati")
    sns.heatmap(comunalitati, cmap="YlGnBu", annot=True, center=0)
    plt.show()

    plt.title("Corelograma Variantelor Specifice")
    sns.heatmap(varianta_specifica, cmap="YlGnBu", annot=True, center=0)
    plt.show()

corelograme_comunalitati_variante_specifice(comunalitati,
varianta_specifica)

# =====
# 9) SCORURI FACTORIALE + SCATTERPLOT
# =====

# Scorurile factoriale: poziția fiecărei observații în spațiul factorilor.
# Aici le calculezi manual ca: Z * loadings (aproximare).
# Notă: unele implementări calculează scorurile cu metode dedicate
# (regresie, Bartlett etc.).
scoruri = pd.DataFrame(
    data_transf @ fa.loadings_,
    index=data.index,
    columns=loadings.columns
)

print(scoruri)

def scatterplot_scoruri(scoruri, c1=0, c2=1):
    x = scoruri.iloc[:, c1]
    y = scoruri.iloc[:, c2]

    plt.title("Scatter Scoruri")
    plt.axhline(y=0, color="gray", linewidth=0.8)
    plt.axvline(x=0, color="gray", linewidth=0.8)

    plt.scatter(x, y, color="blue")

```

```
plt.xlabel(f"{scoruri.columns[c1]}")
plt.ylabel(f"{scoruri.columns[c2]}")
plt.show()

scatterplot_scoruri(scoruri, 0, 1)
```

Discriminanta

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report
from sklearn.preprocessing import StandardScaler

# =====
# 1) FUNCTIE: INLOCUIRE VALORI LIPSA (NaN)
# =====
def inlocuire_nan(df: pd.DataFrame):
    """
    Completează valorile lipsă (NaN) astfel:
    - pentru coloane numerice: cu media coloanei
    - pentru coloane nenumerice: cu moda (cea mai frecventă valoare)

    Atenție: folosește inplace=True (modifică df direct).
    În general merge, dar în unele versiuni de pandas apare
    SettingWithCopyWarning
    dacă df e un view. Aici fiind citit din CSV, de obicei e ok.
    """
    for col in df.columns:
        if df[col].isnull().any():
            if pd.api.types.is_numeric_dtype(df[col]):
                # NaN numeric -> media
                df[col].fillna(df[col].mean(), inplace=True)
            else:
                # NaN categorial -> moda (prima valoare cea mai frecventă)
                df[col].fillna(df[col].mode()[0], inplace=True)

# =====
# 2) FUNCTIA PRINCIPALA
# =====
def main():
    # -----
    # 2.1) CITIRE DATE
    # -----
    # Citim setul de date. index_col=0 presupune ca prima coloană este
    index / ID.
    df = pd.read_csv("hernia.csv", index_col=0)

    # Completăm valorile lipsă
    inlocuire_nan(df)

    # -----
    # 2.2) DEFINIRE TARGET + PREDICTORI
    # -----
```

```

# Presupunem că ultima coloană este targetul (clasa).
# Dacă în fișier targetul e în altă coloană, aici trebuie schimbat.
target_col = df.columns[-1]
predictor_cols = df.columns[:-1]

# X = matricea cu predictorii (variabile explicative)
X = df[predictor_cols].values

# y = vectorul cu clasele (variabila țintă)
y = df[target_col].values

# -----
# 2.3) STANDARDIZARE
# -----
# Standardizare: medie 0, deviație 1.
# NOTĂ IMPORTANTĂ (de examen):
# Cel mai corect e să faci fit pe train și transform pe test (evită
"data leakage").
# Tu ai standardizat înainte de split; e ok pentru laborator, dar nu e
"perfect".
X = StandardScaler().fit_transform(X)

# -----
# 2.4) IMPARTIRE TRAIN / TEST
# -----
# test_size=0.3 => 30% test, 70% train
# random_state => reproducibilitate (același split la fiecare rulare)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)

# =====
# 3) LDA (Linear Discriminant Analysis)
# =====
# LDA găsește axe discriminante (combinații liniare) care separă
clasele.
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)

print("\n=== LDA ===")
print("Clase:", lda.classes_) # clasele întâlnite în y
print("Priors:", lda.priors_) # probabilități a priori ale claselor
(estimate din train)

# -----
# 3.1) SCORURI DISCRIMINANTE PE TEST
# -----
# transform(X_test) -> coordonatele observațiilor pe axe
discriminante.
# nr_axe = min(nr_clase - 1, nr_variabale)
z_test = lda.transform(X_test)

# -----
# 3.2) GRAFICE PENTRU LDA
# -----

# Histogramă pe prima axă discriminantă (dacă există)
if z_test.shape[1] >= 1:
    plt.figure(figsize=(7, 5))

```

```

plt.title("Distribuție scor LDA (axa 1) - set de test")

# Plotează histograme separate pentru fiecare clasă
for clasa in np.unique(y_test):
    plt.hist(
        z_test[y_test == clasa, 0],
        alpha=0.5,
        label=str(clasa),
        bins=10
    )

plt.xlabel("Scor LDA1")
plt.legend()
plt.tight_layout()
plt.show()

# Scatterplot în primele 2 axe discriminante (dacă există 2)
if z_test.shape[1] >= 2:
    plt.figure(figsize=(7, 5))
    plt.title("Instanțe în primele 2 axe discriminante")

    for clasa in np.unique(y_test):
        plt.scatter(
            z_test[y_test == clasa, 0], # axa 1
            z_test[y_test == clasa, 1], # axa 2
            alpha=0.5,
            label=str(clasa)
        )

    plt.xlabel("Scor LDA1")
    plt.ylabel("Scor LDA2")
    plt.legend()
    plt.tight_layout()
    plt.show()

# -----
# 3.3) PREDICTIE + EVALUARE (LDA)
# -----
y_pred_lda = lda.predict(X_test)

# Matrice de confuzie: rând = real, coloană = prezis
cm_lda = confusion_matrix(y_test, y_pred_lda)

# Acuratețe: proporția de clasificări corecte
acc_lda = accuracy_score(y_test, y_pred_lda)

print("\n=== Matrice de confuzie LDA ===")
print(cm_lda)
print(f"Acuratete LDA: {acc_lda*100:.2f}%")

# classification_report: precizie, recall, f1-score pe fiecare clasă
print("Raport clasificare LDA:")
print(classification_report(y_test, y_pred_lda))

# =====
# 4) NAIVE BAYES (GaussianNB)
# =====
# GaussianNB presupune distribuții Gaussiene pentru fiecare variabilă,
pe fiecare clasă.
# E rapid, simplu, uneori surprinzător de bun.
nb = GaussianNB()

```

```

nb.fit(X_train, y_train)

y_pred_nb = nb.predict(X_test)

cm_nb = confusion_matrix(y_test, y_pred_nb)
acc_nb = accuracy_score(y_test, y_pred_nb)

print("\n=== Naive Bayes ===")
print("Matrice de confuzie:")
print(cm_nb)
print(f"Acuratete Bayes: {acc_nb*100:.2f}%")
print("Raport clasificare Bayes:")
print(classification_report(y_test, y_pred_nb))

# =====
# 5) SALVARE REZULTATE (OPTIONAL)
# =====
# Salvăm real vs predicții în CSV, ca să poți analiza manual greșelile.
results_df = pd.DataFrame({
    "Real": y_test,
    "Pred_LDA": y_pred_lda,
    "Pred_Bayes": y_pred_nb
})

results_df.to_csv("Rezultate_Discriminare.csv", index=False)
print("\n=== Rezultate salvate în 'Rezultate_Discriminare.csv' ===")

# =====
# 6) ENTRY POINT
# =====
if __name__ == "__main__":
    main()

```